VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT 2

Deep Learning on Large-Scale Data and Specialized Tasks

Deep Learning on Large-Scale Data for Specialized Computer Vision or Language Tasks

30-class Vietnamese food classification on 30VNFoods
Scratch CNN vs. ImageNet-pretrained ResNet-50, frozen vs. full fine-tuning

GROUP
Group 09 – Ryzen9

INSTRUCTOR
Le Thanh Sach

MEMBERS

Hoang Kim Cuong	2352145
Phan Tan Loc	2352712
Tran Manh Thang	2353124

Table of Contents

01 Problem and Data

02 Exploratory Data Analysis

03 Methodology

04 Results

05 Comparison

06 Error Analysis

07 Conclusion

01

Problem and Data

Problem formulation

Task

Motivation. Delivery, review, and nutrition apps need the dish name from a photo; ImageNet has no Vietnamese dishes.

Task. Single-label, $K = 30$-class classification; one photo is one prediction unit.

$$z = f_\theta(x) \in \mathbb{R}^{30}, \qquad \hat{y} = \arg\max_k z_k$$

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{n}\sum_{i=1}^{n} \log \hat{p}_{i,y_i}$$

$\hat{p} = \mathrm{softmax}(z),\ x \in \mathbb{R}^{3\times 224\times 224},\ y \in \{0, \ldots, 29\}$

Main difficulties

- **Fine-grained:** soups, leaf-wrapped cakes
- **Variation:** region, vendor, plating
- **Clutter:** multi-dish photos
- **Imbalance:** 441–1,517 per class
- **Web data:** cross-label duplicates
- **Compute:** one GPU session per run

Dataset: 30VNFoods

30	24,982	412	4.48 GB
dish classes	images retained	duplicate groups	ZIP archive

Dataset card

Source	Kaggle `quandang/vietnamese-foods` v11
License	CC BY-NC-SA 4.0
Images	25,136 $\rightarrow$ 24,982 retained
Labels	One dish per image (folder)
Approval	**Approved** at M1

Duplicate audit before splitting

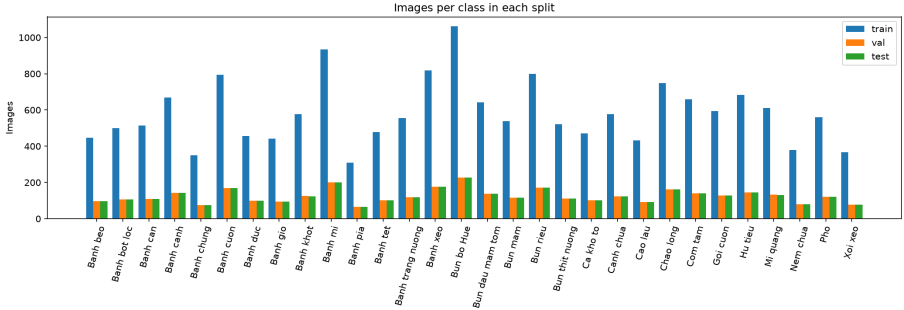
SHA-256 (exact) and dHash ≤ 8 bits (near), checked on thumbnails

-25	redundant exact copies
-79	exact copies, conflicting labels
-50	near copies, conflicting labels

02

Exploratory Data Analysis

Class distribution and imbalance



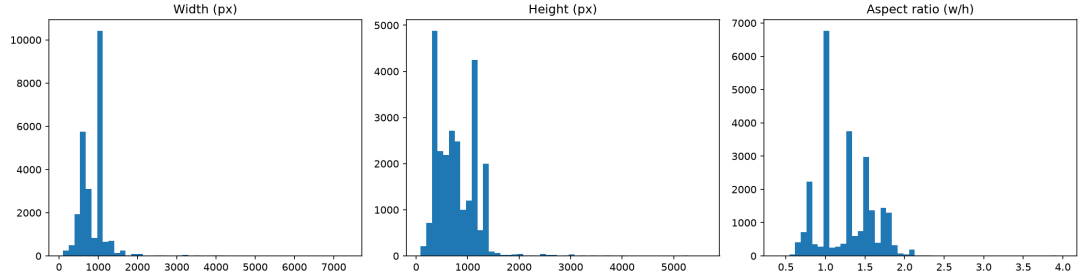
1,517
largest: Bun bo Hue

441
smallest: Banh pia

3.44
imbalance ratio

Macro-F1
primary metric

Input size



1000×720
median size (4:3)

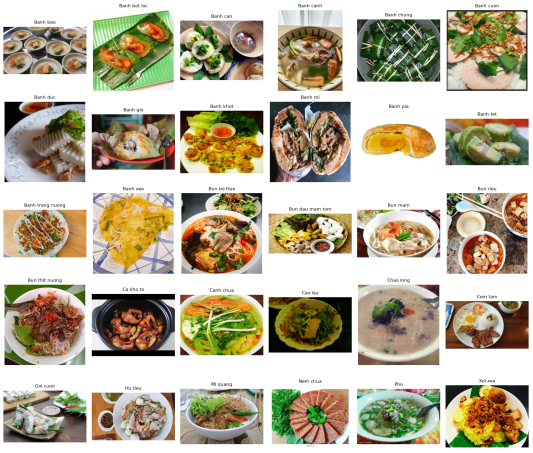
14,180
landscape images

346
short side < 224 px

224×224
model input

Widths span 100–7,360 px, so every image is resized and cropped to 224×224.

Representative samples and visual families



Expected confusions

- Noodle soups share bowls and broths
- Banh chung, Banh tet, Banh gio share banana-leaf wrapping
- Banh khot and Banh can come from similar moulds

Five visual families (analysis only)

- Noodle bowls (10), steamed/leaf-wrapped cakes (7)
- Griddle cakes (4), rice dishes (3), others (6)

Data splitting: leakage control

Group-aware split (seed 42)

- Split unit = **duplicate group**; an image with no match is its own group
- Groups shuffled within each class ⇒ approximately stratified
- All 30 classes and all 24,982 images used

Leakage control

- Official split dropped: **197 groups** and 40 identical files cross it
- No shared IDs, groups, or SHA-256 hashes between splits
- Validation selects; test scored **once** per run

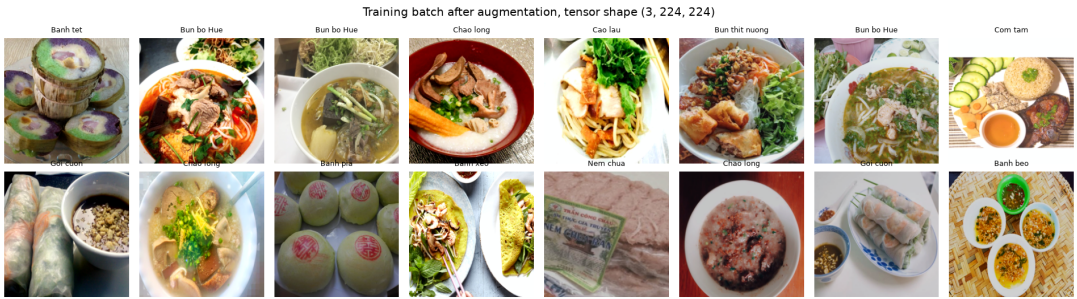
Preprocessing

Transforms

Train: random resized crop 224 (scale 0.7–1.0), flip, colour jitter 0.2
Val/test: resize 232, centre crop 224 (as `IMAGENET1K_V2`)
All splits: ImageNet mean/std normalisation

One batch

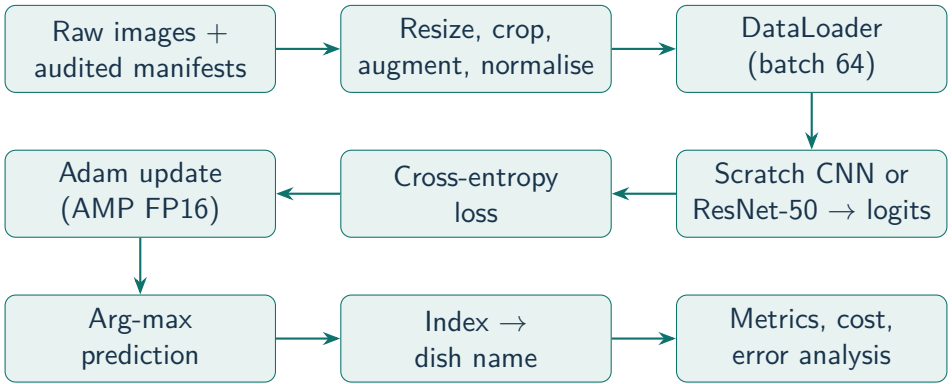
Images $[64, 3, 224, 224]$
Labels $[64]$, logits $[64, 30]$



03

Methodology

Pipeline overview



Only the model and its trainable part change between runs.

Three model configurations

	Scratch CNN	Frozen ResNet-50	Full ResNet-50
Architecture	33 plain convs, GAP, $512 \to 256 \to 30$	Bottleneck residual blocks, GAP, `Linear(2048, 30)`	
Initialisation	Random	`IMAGENET1K_V2` backbone; random head	
Trained part	All layers	Head only	All layers
Total params	21.25 M	23.57 M	23.57 M
Trained params	21.25 M	**61,470**	23.57 M
Learning rate	10^{-3}	10^{-4}	10^{-4}

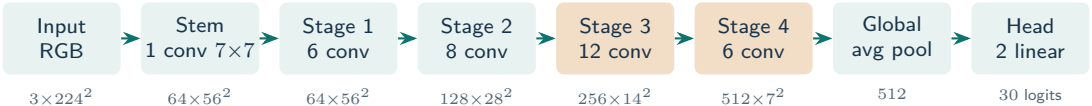
Residual learning

$y = F(x) + x$ keeps deep stacks trainable; the plain CNN has no shortcuts and is hard to optimise.

Fine-tuning regimes

Frozen: backbone in eval mode, BatchNorm keeps ImageNet statistics. **Full:** all layers and BatchNorm adapt.

Scratch CNN: architecture and parameters



Part	Conv / linear	BN	Params
Stem	9,408	128	9,536
Stage 1	221,184	768	221,952
Stage 2	1,105,920	2,048	1,107,968
Stage 3	6,782,976	6,144	6,789,120
Stage 4	12,976,128	6,144	12,982,272
Head	139,038	–	139,038
Total			**21,249,886**

How parameters are counted

Conv: $C_{\text{out}} \times C_{\text{in}} \times k \times k \; (+ C_{\text{out}})$

bias off: BatchNorm follows every conv

BatchNorm: $2C$ (γ, β)

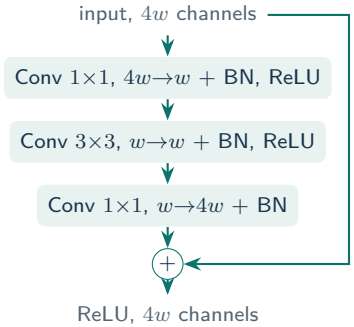
Linear: $M \times N + M$

Example, 3×3 conv $512\to512$:

$$512\times512\times3\times3 = 2{,}359{,}296$$

Stages 3–4 hold **93%** of all parameters.

ResNet-50: bottleneck block and parameters



$$C_{\text{in}}C_{\text{mid}} + C_{\text{mid}}^2k^2 + C_{\text{mid}}C_{\text{out}} + \text{BN}$$

$C_{\text{mid}}{=}w$, $C_{\text{in}}{=}C_{\text{out}}{=}4w$, $k{=}3$:

$$17w^2 + 12w \quad (w{=}64{:}\ 70{,}400)$$

Stage	Blocks	Output	Params
Stem 7×7, 64	–	64×56^2	9,536
Stage 1, $w{=}64$	3	256×56^2	215,808
Stage 2, $w{=}128$	4	512×28^2	1,219,584
Stage 3, $w{=}256$	6	1024×14^2	7,098,368
Stage 4, $w{=}512$	3	2048×7^2	14,964,736
Head $2048\to30$	–	30	61,470
Total	16		**23,569,502**

61,470
trained when frozen (0.26%)

23.57 M
trained in full fine-tuning

Training setup

Shared hyperparameters

Optimizer	Adam, weight decay 0
Scheduler	None (constant LR)
Batch / epochs	64 / max 20
Early stopping	Patience 5, val macro-F1
Checkpoint	Best val macro-F1
Regularisation	Augmentation only
Seed	42 everywhere
Precision	AMP FP16 train, FP32 eval

Environment

- Kaggle Tesla T4 (14.6 GB), 4 CPU cores
- PyTorch 2.11, TorchVision 0.26, CUDA 13.0

Safeguards

- Reloaded checkpoint must reproduce its saved val metrics
- Test evaluated **once** per run
- No hyperparameter search

Experimental design

	A: baseline vs. pretrained	**B: frozen vs. full (controlled ablation)**
Question	Does starting from ImageNet weights beat training from scratch?	Does also training the backbone beat keeping it frozen?
Changed	Architecture + initialisation (+ LR)	`freeze_backbone` only
Fixed	Manifests, seed, augmentation, loss, optimizer, batch, budget, early stopping, AMP, GPU	All of A + architecture, weights, head init, LR 10^{-4}
Metrics	Val/test macro-F1 (primary), accuracy	Same + trained params, train time, inference cost
Comparison	Δ val macro-F1 = frozen − CNN	Δ val macro-F1 = full − frozen

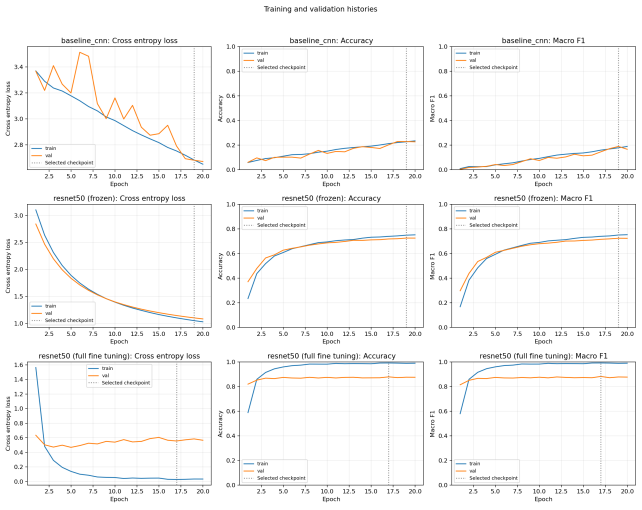
How we compare

Compare on validation, confirm on test; A shows *whether* pretraining helps, not *which factor*.

04

Results

Training behaviour



Scratch CNN: underfits

Train/val accuracy both ∼23%, still rising at epoch 20.

Frozen ResNet-50: smooth

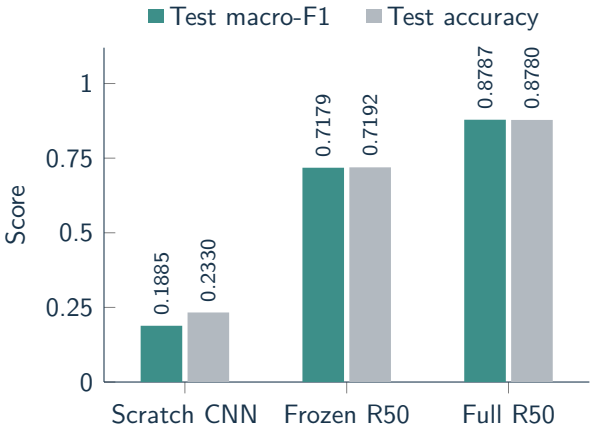
Val macro-F1 0.298 → 0.724; small gap; still improving.

Full ResNet-50: fast, then overfits

0.815 after epoch 1, plateau ∼0.87 from epoch 3; train acc 99.2%, val loss rises after epoch 5.

Selected epochs: 19, 19, 17.

Test performance



0.8787
best test macro-F1

3,290
of 3,747 correct

Validation and test agree

	Val F1	Test errors
Scratch CNN	0.1898	2,874
Frozen R50	0.7245	1,052
Full R50	**0.8824**	**457**

Val–test gap below 0.007; accuracy is close to macro-F1 for both ResNets.

One run per configuration (seed 42), so no mean ± std is reported.

Compute cost on one Tesla T4

	CNN	Frozen	Full
Trained params	21.25 M	0.06 M	23.57 M
Checkpoint (MB)	243.4	90.7	270.2
Training time (min)	65.2	55.5	60.5
Train throughput (img/s)	127.4	127.7	116.4
Latency b1 AMP (ms)	3.24	6.73	5.88
Throughput b64 AMP (img/s)	1,534	807	804
Peak memory b64 FP32 (MB)	521	777	777

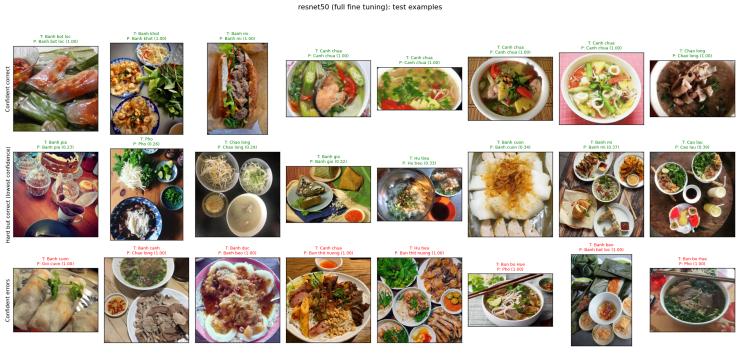
383×
more trained params, full vs. frozen

+9%
training time for full fine-tuning

≈ 0
inference cost difference

Data loading dominates epoch time. CNN is fastest, but 0.19 macro-F1 makes its speed useless. Model-only timing: synthetic inputs, 10 warm-up + 50 timed iterations; shared Kaggle GPU, so timings are indicative.

Qualitative results (full ResNet-50)



Confident correct

Close-ups of a single dish.

Unsure but correct

Table scenes with several dishes.

Confident wrong

Similar dishes (Bun bo Hue / Pho), mixed platters.

05

Comparison

Hypothesis results

A CNN → frozen ResNet-50

Does ImageNet pretraining beat training from scratch?

Scratch CNN **0.1898** → Frozen R50 **0.7245**

Δ val macro-F1 $= 0.7245 - 0.1898$

+0.5347

YES

B frozen → full ResNet-50

Does fine-tuning the backbone beat freezing it?

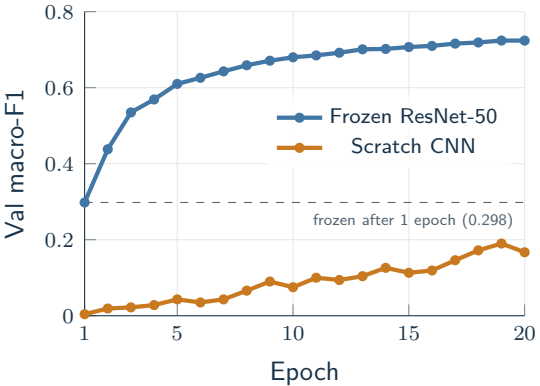
Frozen R50 **0.7245** → Full R50 **0.8824**

Δ val macro-F1 $= 0.8824 - 0.7245$

+0.1579

YES

CNN vs. frozen (1/2): no prior knowledge



Frozen: strong from epoch 1

After just **1 epoch** (0.298) it already beats the CNN after **20 epochs** (0.190).

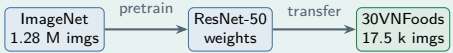
CNN: underfits

Even on its own training images it gets only 23.5% right. Loss drops from 3.37 (random guessing) to just 2.65.

Pretrained features give frozen a head start that the CNN cannot catch up within 20 epochs.

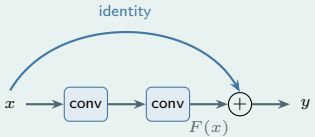
CNN vs. frozen (2/2): pretraining & skip connections

Transfer learning from ImageNet



- Backbone pretrained on **1.28 M** ImageNet images: already encodes edges, textures, object parts
- Frozen trains only a **61k**-weight linear head
- CNN learns **21.25 M** weights from scratch on only **17,486** images

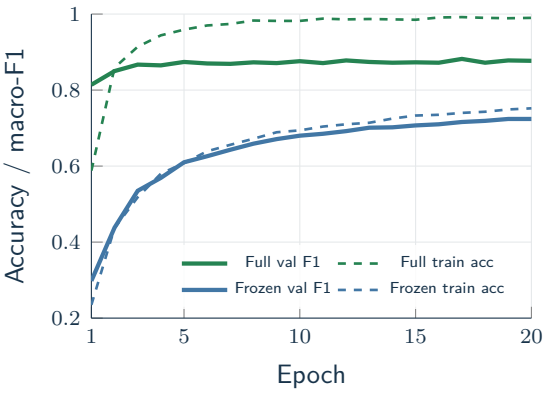
Residual (skip) connections



- $y = F(x) + x$: gradients flow through the identity path to early layers
- CNN: **33 plain conv layers**, no shortcuts, hard to optimise
- Evidence: CNN **underfits** (train acc only 23.5%)

CNN loses: it lacks both pretrained features and skip connections.

Frozen vs. full (1/2): limited capacity



Frozen: linear probe, high bias

Train accuracy plateaus at **75.2%**: it **underfits**. A linear head cannot separate classes that are not linearly separable in the fixed feature space.

Full: higher capacity, lower bias

End-to-end optimisation enlarges the hypothesis space: **1 epoch** (0.814) beats frozen's 20 epochs (0.724).

Frozen's gap is bias (underfitting), not variance or training time.

Frozen vs. full (2/2): domain shift

Frozen: fixed ImageNet features

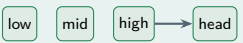
0.06 M trainable weights

backbone fixed (no gradient)

- **Domain shift**: few ImageNet classes are dishes
- Deep features are **source-specific** (Yosinski et al., 2014)
- A linear head cannot create missing cues

Full: features adapt to our food

23.57 M trainable weights



all layers updated

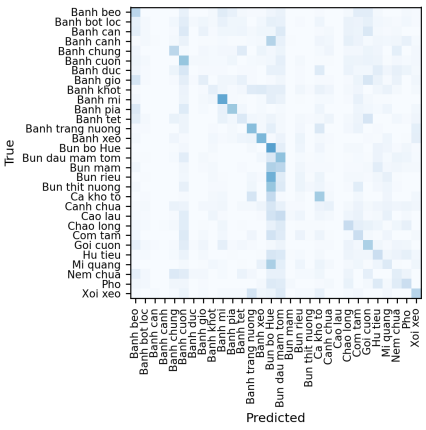
- ImageNet weights as a **prior**; high-level layers re-specialised
- Small LR (10^{-4}) learns fine-grained cues without catastrophic forgetting
- Controlled: **only** backbone updates differ

Fine-tuning closes the domain gap by adapting the features themselves.

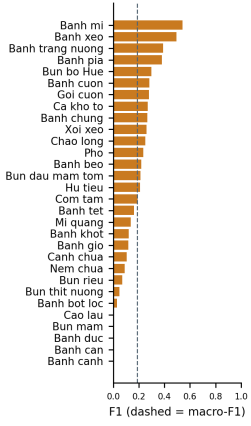
06

Error Analysis

Scratch CNN: where the errors are



Confusion matrix (row-normalised)



Per-class F1, red = weakest

- **Error rate:** 76.7% (2,874 images)
- **Weakest: 5 classes with F1 = 0**, e.g. Banh canh, Banh duc
- **Vertical stripes:** many classes are predicted as **Bun bo Hue** (505 errors)

Predictions collapse onto a few classes.

Scratch CNN: misclassified examples

Legend

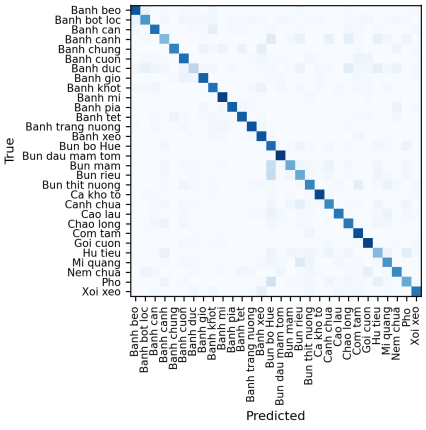
T: true

P: predicted

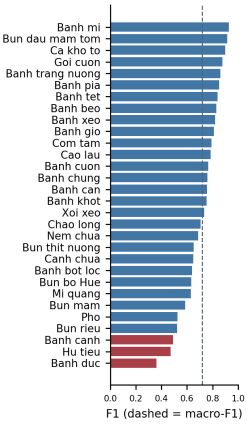
conf: confidence

Low-confidence guesses, often across dish families: the model sees colour and texture, not the dish.

Frozen ResNet-50: where the errors are



Confusion matrix (row-normalised)

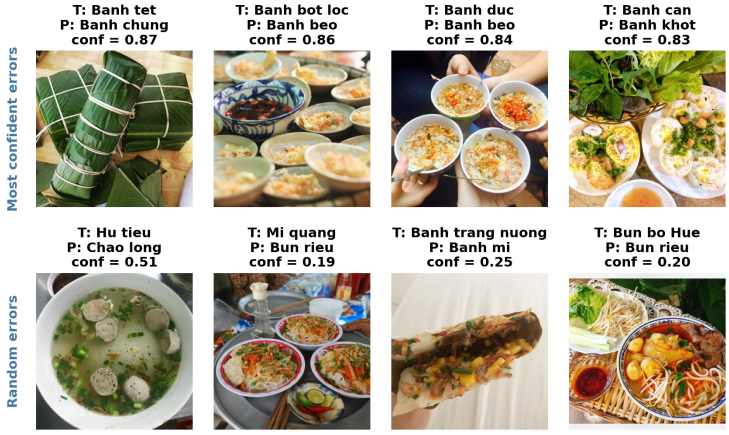


Per-class F1, red = weakest

- **Error rate:** 28.1% (1,052 images)
- **Weakest: Banh duc 0.36**, Hu tieu 0.47, Banh canh 0.49
- **Main confusion:** inside the **noodle-soup** group, e.g. Bun rieu → Bun bo Hue (42)

Clear diagonal; errors stay within similar dishes.

Frozen ResNet-50: misclassified examples



Legend

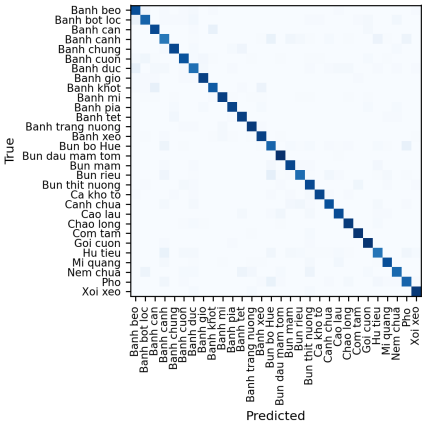
T: true

P: predicted

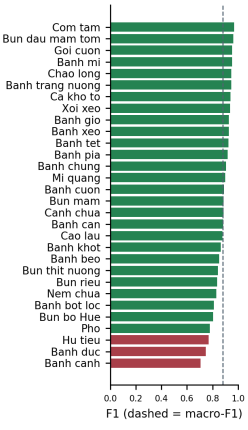
conf: confidence

Right family, wrong dish (Banh tet → Banh chung): fixed ImageNet features miss fine details.

Full ResNet-50: where the errors are



Confusion matrix (row-normalised)



Per-class F1, red = weakest

- **Error rate:** 12.2% (457 images)
- **Weakest: Banh canh 0.71**, Banh duc 0.75, Hu tieu 0.77
- **Main confusion:** lookalike soups in **both directions**, e.g. Bun bo Hue $\leftrightarrow$ Pho

Remaining errors reflect real visual similarity.

Full ResNet-50: misclassified examples

Legend

T: true

P: predicted

conf: confidence

Errors come from lookalikes, mixed plates and unusual views, but often with confidence near 1.0 (overconfident).

Comparison across the three models

Test F1	CNN	Frozen	Full
Banh can	0.00	0.76	0.88
Banh canh	0.00	0.49	**0.71**
Banh duc	0.00	**0.36**	0.75
Hu tieu	0.21	0.47	0.77
Macro-F1	0.19	0.72	0.88
Test errors	2,874	1,052	457

CNN → frozen: recognises dishes

Biggest gains on classes the CNN never got right: Cao lau +0.78, Banh can +0.76.

Frozen → full: separates details

Biggest gains on similar-looking dishes: Banh duc +0.39, Bun rieu +0.31.

Errors shift from wrong family to lookalike dishes; Banh canh, Banh duc and Hu tieu stay hardest.

07

Conclusion

Limitations

Training budget

The scratch CNN had **not converged** and still underfits. The shared training budget favours pretrained models, so the gain of pretraining may be **overstated**.

Experimental design

Experiment A changes pretraining, architecture and learning rate **at once**, so the effect of pretraining alone is not isolated. Each configuration ran with a **single seed**, so variance is unknown.

Regularisation and calibration

Full fine-tuning **overfits** without weight decay or early stopping. It is also **overconfident**: many of its errors are made with high confidence.

Input and task setup

At 224 px, the **fine details** that separate lookalike soups are lost. A single label per image cannot describe **multi-dish** photos.

Conclusion and future work

BEST

0.190 Scratch CNN

0.725 Frozen ResNet-50

0.882 Full ResNet-50

Validation macro-F1

Pretraining matters most; adapting the backbone gives a clear further gain.

Future work

Fairer baseline

Train the CNN longer with a tuned learning rate; add a ResNet-50 trained from scratch.

Reliable results

Run several seeds; add weight decay and early stopping to full fine-tuning.

Finer recognition

Higher input resolution, calibrated confidence, and top-3 predictions.

Thank you